```
$ curl -b ~/.curl_cookies -c ~/.curl_cookies http://localhost:8080/
index.php
Session still active with ID: 54a9a1f51c7842c0a58d0b971cb463d1

# 30 seconds later...

$ $ curl -b ~/.curl_cookies -c ~/.curl_cookies http://localhost:8080/
index.php
Session expired! Regenerated with new ID: e59d41cd5d48fd347ce5ece6e6a944e2
```

If you feel inclined, you can check the contents of the session save path (defaults to `/tmp`) inside your cli-server container; you'll notice `obsolete|b:1` appears in expired files. An attempt to access an expired session ID should be viewed with suspicion (after all, the client should now be using the new ID), but bear in mind that there could be benign reasons for a legitimate user to make a request with an old session ID (e.g., switching devices). Note that `session_regenerate_id()` accepts a boolean argument: `session_regenerate_id(true);` would trigger the immediate removal of the old session ID, but this can cause problems for legitimate users. Typically, you would preserve the old session data and rely on PHP's session garbage collection mechanism to clean up old session data, which we'll look at in a moment.

Careful with That Cookie

There are other ways for bad actors to control your users' session IDs. If the attacker can't trick your users into asking for specific IDs via a URL, they can still utilize other methods. Cookies can be set via JavaScript's `document.cookie` object and HTML's `<meta http-equiv="Set-Cookie">` tag. Both of these methods will require some form of injection vulnerability, which would have to be within the domain under attack (i.e., it's "your" injection vulnerability). If a request has a session ID cookie set and that session does not exist, it should probably be treated as a potential attack. Depending on the session module in use (files, memcached, Redis), you could have a dig around in the data store and validate that the session is known, which means searching a filesystem or some other equivalent API call.

Alternatively, PHP's session management will do the work for you. As mentioned already, the `session.use_strict_mode = 1` ini directive will ensure that an unknown session ID will be discarded. Even better, you can enforce strictness at the moment the session starts by passing some information to `session_start()`: